

Experiment-11

Write the Python Program for map Colouring to Implement CSP

AIM

To implement the Map Coloring Problem using Constraint Satisfaction Problem (CSP) in Python using the backtracking technique.

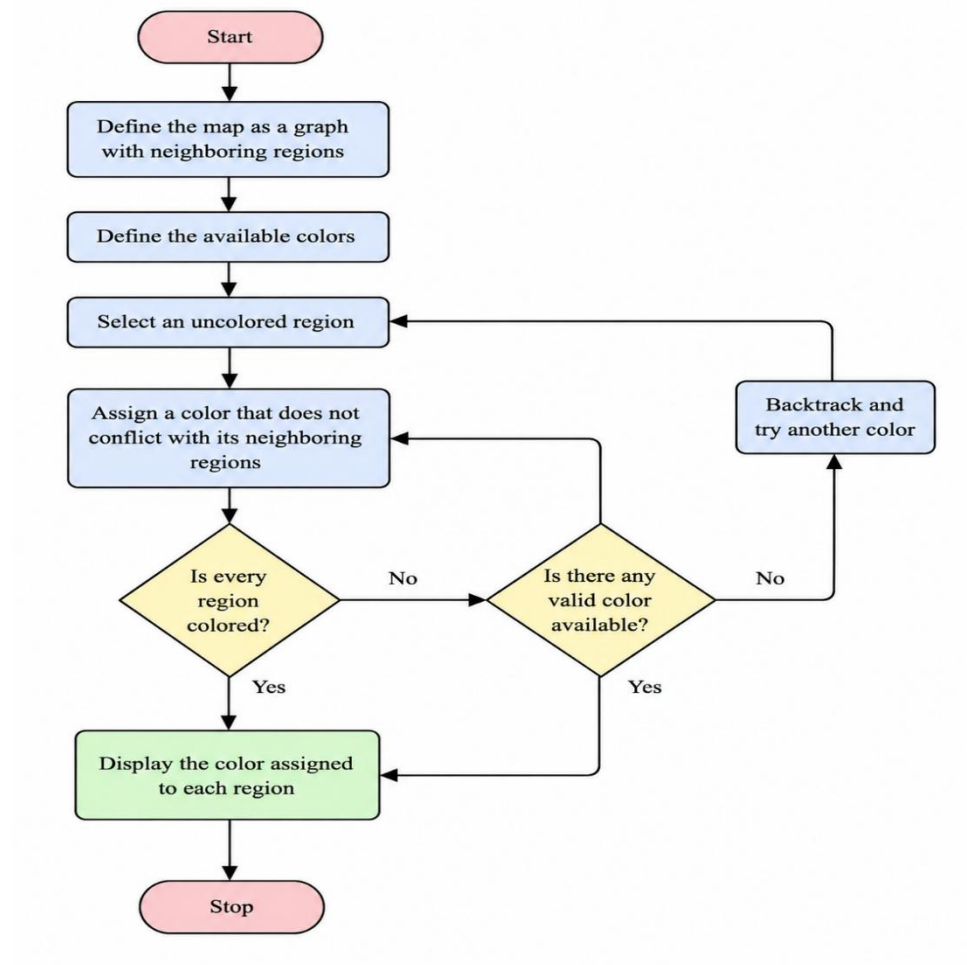
Objective

- To implement the Map Coloring Problem using CSP.
- To assign colors to map regions while satisfying the given constraints.
- To understand Constraint Satisfaction Problems in Artificial Intelligence.

Algorithm

- Start the program.
- Define the map as a graph with neighboring regions.
- Define the available colors.
- Select an uncolored region.
- Assign a color that does not conflict with its neighboring regions.
- If no valid color is available, backtrack and try another color.
- Repeat until all regions are colored.
- Display the color assigned to each region.
- Stop the program.

Flowchart:



Code:

Map Coloring using CSP

```
def color_map(graph, colors, result, node):
```

```
    if node == len(graph):
```

```
        return True
```

```
    place = list(graph)[node]
```

```
    for color in colors:
```

```
if all(result.get(n) != color for n in graph[place]):
```

```
    result[place] = color
```

```
    if color_map(graph, colors, result, node + 1):
```

```
        return True
```

```
    del result[place]
```

```
return False
```

```
graph = {
```

```
    'A': ['B', 'C'],
```

```
    'B': ['A', 'C', 'D'],
```

```
    'C': ['A', 'B', 'D'],
```

```
    'D': ['B', 'C']
```

```
}
```

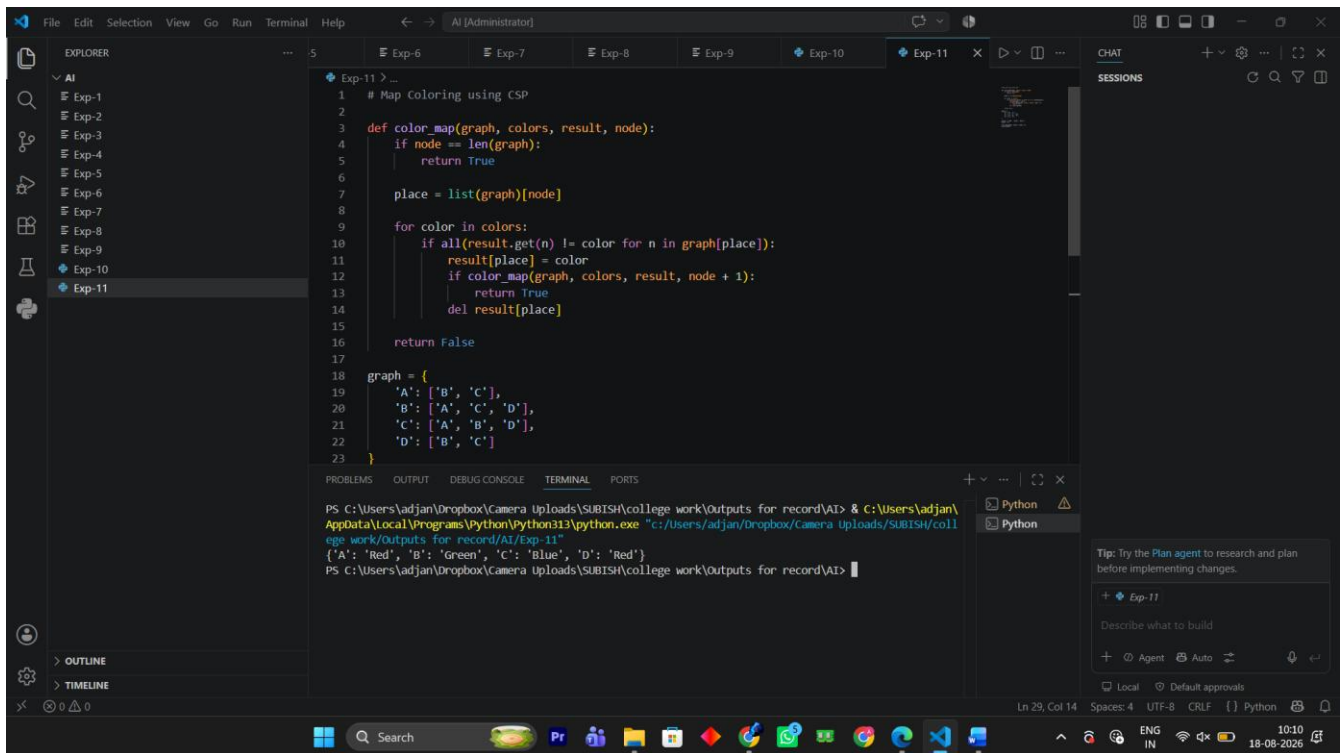
```
colors = ['Red', 'Green', 'Blue']
```

```
result = {}
```

```
color_map(graph, colors, result, 0)
```

```
print(result)
```

Output:



```
1 # Map Coloring using CSP
2
3 def color_map(graph, colors, result, node):
4     if node == len(graph):
5         return True
6
7     place = list(graph)[node]
8
9     for color in colors:
10        if all(result.get(n) != color for n in graph[place]):
11            result[place] = color
12            if color_map(graph, colors, result, node + 1):
13                return True
14            del result[place]
15
16    return False
17
18 graph = {
19     'A': ['B', 'C'],
20     'B': ['A', 'C', 'D'],
21     'C': ['A', 'B', 'D'],
22     'D': ['B', 'C']
23 }
```

PS C:\Users\adjan\Dropbox\Camera Uploads\SUBISH\college work\Outputs for record\AI> & C:\Users\adjan\AppData\Local\Programs\Python\Python313\python.exe "C:/Users/adjan/Dropbox/Camera Uploads/SUBISH/college work/Outputs for record/AI/Exp-11"

{'A': 'Red', 'B': 'Green', 'C': 'Blue', 'D': 'Red'}

PS C:\Users\adjan\Dropbox\Camera Uploads\SUBISH\college work\Outputs for record\AI>

Result:

The Map Coloring Problem was successfully implemented using Constraint Satisfaction Problem (CSP) and valid colors were assigned to all regions without conflicts.

Conclusion:

The Map Coloring problem was successfully implemented using CSP. The program assigns colors to different regions while satisfying the constraint that adjacent regions must have different colors.